Problem Sheet 09

Question:

A farmer wants to cross a river carrying a wolf, a goat, and a cabbage. The boat can carry the farmer and **only one** item at a time. If the **wolf is left with the goat**, the wolf eats the goat. If the **goat is left with the cabbage**, the goat eats the cabbage.

A Prolog program is written to represent this problem. The state is represented as
`state(Farmer, Wolf, Goat, Cabbage)`
where each value is either **east** or **west**.

The program defines unsafe states, travel rules, and a recursive search algorithm to find the solution.

Explain the purpose of each part of the given Prolog code and describe how the program finds a safe sequence of moves to reach the goal state.

1. Initial and Goal States

```
initial_state(state(east,east,east,east)).
goal_state(state(west,west,west,west)).
```

✓ Represents positions of **Farmer, Wolf, Goat, Cabbage**
✓ `east = starting side`
✓ `west = destination`
✓ The goal is to move all four from east → west

---

2. Defining Unsafe States

```
unsafe(state(B,A,A,_)):- notequal(A,B).
unsafe(state(B,A,_,A)):- notequal(A,B).
```

These rules check when a state is **dangerous**:

Rule 1 — Wolf eats goat

```
unsafe(state(B,A,A,_))
```

Meaning: goat and wolf are together (A,A), farmer is away (B ≠ A)

Rule 2 — Goat eats cabbage

```
unsafe(state(B,A,_,A))
```

Meaning: goat and cabbage are together (A,A), farmer is away (B ≠ A)

The predicate:

```
notequal(east,west).
notequal(west,east).
```

Ensures Prolog understands "east ≠ west".

---

3. Printing Move Descriptions

These clauses print readable English descriptions of the moves:

```
write_action(move1(P1,P2)) :- write('Farmer goes with goat from '),
write(P1), write(' to '), write(P2), nl.
```

```
write_action(move2(P1,P2)) :- write('Farmer goes alone from '), write(P1),
write(' to '), write(P2), nl.
write_action(move3(P1,P2)) :- write('Farmer goes with wolf from '),
write(P1), write(' to '), write(P2), nl.
write_action(move4(P1,P2)) :- write('Farmer goes with grass from '),
write(P1), write(' to '), write(P2), nl.
```

Each move is labelled (goat = move1, alone = move2, wolf = move3, cabbage = move4).

---

4. Travel Rules (State Transitions)

These predicates define **how the farmer moves** and **how the state updates**.

4.1 Farmer takes the goat

```
travel(state(P1,P1,A,B),move1(P1,P2),state(P2,P2,A,B)) :-
  notequal(P1,P2),
  not(unsafe(state(P2,P2,A,B))).
```

Conditions:

- Farmer (P1) and goat (P1) are on same side
- Move to P2
- New state must be safe

4.2 Farmer goes alone

```
travel(state(P1,A,B,C),move2(P1,P2),state(P2,A,B,C)) :-
  notequal(P1,P2),
  not(unsafe(state(P2,A,B,C))).
```

Only farmer moves; animals remain.

4.3 Farmer takes the wolf

```
travel(state(P1,A,P1,B),move3(P1,P2),state(P2,A,P2,B)) :-
  notequal(P1,P2),
  not(unsafe(state(P2,A,P2,B))).
```

Wolf must be on same side as farmer.

4.4 Farmer takes the cabbage

```
travel(state(P1,A,B,P1),move4(P1,P2),state(P2,A,B,P2)) :-
   notequal(P1,P2),
   not(unsafe(state(P2,A,B,P2))).
```

5. Writing the Action List

```
write_action_list([]).
write_action_list([H|T]) :- write_action(H), write_action_list(T), !.
```

Recursively prints every move in the final solution list.

6. Depth-First Search Algorithm

The key solving predicate:

```
can(S, S, _, []).
```

If current state equals goal → no more actions.

```
can(S1, S2, V, [A|L]) :-
   travel(S1, A, T),
   not(member(T, V)),
   can(T, S2, [T|V], L).
```

This means:

1. Try a valid move **A** from state **S1** to **T**
2. Ensure **T** is not already visited (avoid loops)
3. Continue searching until goal state (**S2**) is reached
4. Build the action list as [A | L]

7. Running the Program

```
go :-
```

```
initial_state(S),
goal_state(G),
can(S, G, [S], L),
write_action_list(L), !.
```

Steps:

1. Start from initial state
2. Try to reach goal state
3. Avoid revisiting states
4. Get the action list
5. Print each move
6. Stop at the first valid solution (!)